



Toolchain optimizations





Best toolchain for your project

Optimizing the cross-compiling toolchain is typically one of the first things to do:

- ▶ The benefits of a toolchain change will be more significant and easier to measure if other optimizations haven't been done yet.
- ▶ Here's what you can change in a toolchain, with a potential impact on boot time, performance and size:
 - Components: versions of *gcc* and *binutils* More recent versions can feature better optimization capabilities.
 - C library: *glibc*, *uClibc*, *musl* *uClibc* and *musl* libraries make a smaller root filesystem
 - Instruction set variant: *ARM* or *Thumb2* (on 32 bit only), *Hard Float* support or not. Can have an impact on code performance and code size.
 - *Thumb2*, available only on ARM 32, encodes the same instructions as *ARM* but in a more compact way, at least significantly reducing size.
 - *ARM EABIhf*, in addition to being more efficient, also reduces code size compared to *ARM EABI*, but only on binaries doing floating point computation. For example, [libavcodec](#) size is only reduced by 4K (-0.03%). That's negligible.



Choosing the C library

- ▶ The C library is hardcoded at toolchain creation time
- ▶ Available C libraries:
 - *glibc*: most standard and featureful
 - *uClibc*: smaller and configurable. Has been around for about 20 years.
 - *musl*: an alternative to *uClibc*, developed more recently but mature too.